

Week12

Task1:

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 20

int *buffer;
int buffer_size;
int in = 0;
int out = 0;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;
void *producer(void *arg){
    int i;
    for (i = 1; i <= ITEMS; i++){
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Producer produced: %d\n", i);
        in = (in + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}
void *consumer(void *arg){
    int i;
    int item;
    for (i = 1; i <= ITEMS; i++){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
    }
}
```

```

        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        usleep(150000);
    }
    return NULL;
}

int main(){
    pthread_t producer_thread;
    pthread_t consumer_thread;
    printf("Enter buffer size: ");
    scanf("%d", &buffer_size);
    if (buffer_size <= 0){
        printf("Invalid buffer size.\n");
        return 1;
    }
    buffer = malloc(buffer_size * sizeof(int));
    if (buffer == NULL){
        printf("Memory allocation failed.\n");
        return 1;
    }
    sem_init(&empty, 0, buffer_size);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    printf("\nBuffer size: %d\n", buffer_size);
    printf("Items to produce: %d\n\n", ITEMS);
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);
    printf("\nProducer-Consumer execution completed.\n");
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    free(buffer);
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./producer
Enter buffer size: 2

Buffer size: 2
Items to produce: 20

Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Consumer consumed: 6
Producer produced: 8
Consumer consumed: 7
Producer produced: 9
Consumer consumed: 8
Producer produced: 10
Consumer consumed: 9
Producer produced: 11
Consumer consumed: 10
Producer produced: 12
Consumer consumed: 11
Producer produced: 13
Consumer consumed: 12
Producer produced: 14
Consumer consumed: 13
Producer produced: 15
Consumer consumed: 14
Producer produced: 16
Consumer consumed: 15
Producer produced: 17
Consumer consumed: 16
Producer produced: 18
Consumer consumed: 17
Consumer consumed: 15
Producer produced: 17
Consumer consumed: 16
Producer produced: 18
Consumer consumed: 17
Producer produced: 19
Consumer consumed: 18
Producer produced: 20
Consumer consumed: 19
Consumer consumed: 20

Producer-Consumer execution completed.
root@DESKTOP-VMN932K:~#

```

Task2:

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;
void *thread1_deadlock(void *arg){
    printf("Thread 1: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Resource 1 locked\n");
    sleep(1);
    printf("Thread 1: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Resource 2 locked\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    return NULL;
}
void *thread2_deadlock(void *arg){
    printf("Thread 2: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Resource 2 locked\n");
    sleep(1);
    printf("Thread 2: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Resource 1 locked\n");
    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
    return NULL;
}
void *thread1_ordered(void *arg){
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Both resources acquired\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    return NULL;
}

```

```

    return NULL;
}
void *thread2_ordered(void *arg){
    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Both resources acquired\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    return NULL;
}
int main(){
    pthread_t t1, t2;
    printf("==== DEADLOCK DEMONSTRATION ==== \n\n");
    pthread_create(&t1, NULL, thread1_deadlock, NULL);
    pthread_create(&t2, NULL, thread2_deadlock, NULL);
    sleep(3);
    printf("\nBoth threads are waiting for each other.\n");
    printf("Deadlock has occurred.\n");
    pthread_cancel(t1);
    pthread_cancel(t2);
    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);
    printf("\n==== DEADLOCK PREVENTION ==== \n\n");
    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);
    pthread_create(&t1, NULL, thread1_ordered, NULL);
    pthread_create(&t2, NULL, thread2_ordered, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("\nDeadlock prevented using resource ordering.\n");
    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./deadlock
==== DEADLOCK DEMONSTRATION ====

Thread 2: Trying to lock Resource 2
Thread 2: Resource 2 locked
Thread 1: Trying to lock Resource 1
Thread 1: Resource 1 locked
Thread 1: Trying to lock Resource 2
Thread 2: Trying to lock Resource 1

Both threads are waiting for each other.
Deadlock has occurred.

==== DEADLOCK PREVENTION ====

Thread 1: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Both resources acquired
Thread 2: Locking Resource 1
Thread 2: Locking Resource 2
Thread 2: Both resources acquired

Deadlock prevented using resource ordering.
root@DESKTOP-VMN932K:~#

```